

Phase 1 压测报告 — 初始基线 + 高限流重测

日期: 2026-06-21 | 环境: Docker Compose 5 容器 (8.163.136.37) | 2 轮压测 | 测试用户: 50~200 个

目录

- [1. 环境状态](#)
- [2. 初测: 低限流基线](#)
- [3. 重测: 高限流真实容量](#)
- [4. 瓶颈演讲与结论](#)

1. 环境状态

组件	状态
seckill-backend	healthy
seckill-mysql	healthy
seckill-redis	healthy (已预热 20 商品)
seckill-rabbitmq	healthy (4 队列空)
seckill-frontend	healthy

2. 初测: 低限流基线 (100/s)

2.1 限流配置 (初始值)

接口	限流值	注解位置
/product/detail	500/s	ProductController.java:58
/product/active	300/s	ProductController.java:78
/order	100/s	OrderController.java:28

2.2 单接口基线

/product/detail (GET, 限流 500/s)

并发	总请求	成功	失败	成功QPS	P50(ms)	P90(ms)	P99(ms)
1	10	10	0	284	2.9	4.4	6.6
10	100	100	0	623	12.4	20.2	28.8
50	500	421	79	498	39.7	97.7	171.3
100	1000	944	56	519	75.9	153.4	231.4
200	2000	1839	161	508	67.3	148.9	262.7
500	5000	4158	842	502	103.5	238.5	384.2

/product/active (POST, 限流 300/s)

并发	总请求	成功	失败	成功QPS	P50(ms)	P90(ms)	P99(ms)
1	10	10	0	154	6.0	7.2	7.3
10	100	100	0	361	23.0	32.0	47.2
50	500	404	96	372	74.9	121.9	185.6
100	1000	579	421	295	97.2	190.5	289.4
200	2000	1198	802	302	179.2	362.0	581.6
500	5000	2848	2152	300	142.9	318.3	533.8

/order (POST, 限流 100/s) ⚠ 瓶颈

并发	总请求	成功	失败	成功QPS	P50(ms)	P90(ms)	P99(ms)
1	10	10	0	63	15.8	17.1	18.7
10	100	40	60	99	33.2	61.8	86.2
50	200	0	200	0	32.8	57.2	72.5
100	200	0	200	0	56.3	103.1	128.9
200	200	0	200	0	34.2	56.8	82.8

2.3 分层验证

限流层

- 拦截次数: 5,115
- 日志条数: 10,230
- TTL 回收: 正常

Redis+Lua 层

- 库存扣减成功: 51 次
- Redis/DB 偏差: 0 (完全一致)
- Lua 脚本均正常返回

DB 层

- 超卖检查: 0 超卖
- 一人多单: 0 违规
- remaining + orders = initial 完全吻合

MQ 层

- 队列积压: 0
- 死信队列: 0
- 消费者: 4/4 在线

2.4 瓶颈总结

瓶颈	严重度	说明
/order 100/s 限流	严重	c≥50 时成功率 0%，秒杀接口形同虚设
/product/active P99 > 500ms	中等	c=200 时突破 500ms
限流器非滑动窗口	中等	INCR+固定TTL 存在边界误杀

3. 重测：高限流真实容量 (1000/s)

测试用户: ptest001-200 (200 个) | 限流调整: detail 500→2000, active 300→1000, order 100→1000

3.1 限流配置对比

接口	初始值	重测值	变更
/product/detail	500/s	2000/s	4x
/product/active	300/s	1000/s	3.3x
/order	100/s	1000/s	10x

3.2 高限流基线 vs 初始基线

/product/detail (GET)

并发	成功QPS(初)	成功QPS(重)	P99(初)	P99(重)	失败率(初)	失败率(重)
1	284	170	6.6	8.0	0%	0%
10	623	456	28.8	37.0	0%	0%
50	498	520	171.3	182.1	15.8%	0%
100	519	558	231.4	236.2	5.6%	0%
200	508	542	262.7	272.5	8.1%	0%
500	502	573	384.2	420.7	16.8%	0%

结论: 真实容量 ~**570 QPS**，P99 在 c=500 时 < 421ms。初测的 15%+ 失败率全部消失。

/product/active (POST)

并发	成功QPS(初)	成功QPS(重)	P99(初)	P99(重)	失败率(初)	失败率(重)
1	154	58	7.3	71.4	0%	0%
10	361	292	47.2	61.7	0%	0%
50	372	421	185.6	168.2	19.2%	0%
100	295	493	289.4	315.0	42.1%	0%
200	302	535	581.6	462.2	40.1%	0%
500	300	540	533.8	1308	43.0%	0%

结论: 真实容量 ~**540 QPS**。P99 在 c=500 时跳升到 1308ms——DB 查询层成为瓶颈。失败率从 43% 降到 0%。

/order (POST — 秒杀核心)

并发	成功QPS(初)	成功QPS(重)	P50(初)	P50(重)	主要失败原因(初)	主要失败原因(重)
1	63	34	15.8	21.2	—	—
10	99	135	33.2	57.6	限流拦截	重复购买
50	0	109	32.8	96.3	限流拦截	重复购买
100	0	0	56.3	93.2	限流拦截	重复购买
200	0	0	34.2	67.6	限流拦截	重复购买

结论: 瓶颈从 "限流拦截" 转移为 "uk_user_goods 重复购买检查"。真实吞吐 ~135 QPS (c=10 时)。c≥100 后因 200 用户×20 商品的唯一组合耗尽，全部请求被"已购买过该商品"拒绝——这不是系统 Bug，而是业务规则正确执行的结果。

3.3 重测数据一致性

检查项	结果
超卖检查	☒ 0 超卖 (<code>remaining + orders = initial</code> 全部吻合)
Redis/DB 库存偏差	☒ 0 (完全一致)
一人多单	☒ 0 违规
MQ 队列积压	☒ 0
限流拦截	☒ 0 (相比初测 5,115 次, 证明限流是旧瓶颈)
Lua 脚本执行	☒ 200 次全部成功

4. 瓶颈演进与结论

4.1 最终瓶颈总结

瓶颈	严重度	说明	初测状态
<code>uk_user_goods</code> 唯一约束	● 业务特征	用户+商品组合有限时, 并发下单自然耗尽	被限流掩盖
<code>/product/active</code> DB 查询	● 中等	c=500 时 P99 飙到 1308ms, DB <code>listActiveProducts</code> 需缓存	被限流掩盖
<code>/product/detail</code> P99 上升	● 轻微	P99 420ms @ 570QPS, 仍在可接受范围	接近
Redis+Lua 原子操作	☒ 验证通过	200 并发扣减无超卖, 偏差 0	☒
MQ 异步削峰	☒ 验证通过	队列 0 积压, 消费速度匹配生产速度	☒

4.2 关键结论

初测结论（低限流）	重测结论（高限流）
<code>/order 100/s</code> 限流是最大瓶颈， <code>c≥50</code> 时成功率 0%	限流已不是瓶颈
系统看似"挂了"，其实是限流器在保护	真实系统容量被暴露
无法评估真实系统容量	<code>/order</code> ~135 QPS
无法发现 DB 查询层的性能问题	<code>/product/active</code> ~540 QPS
	<code>/product/detail</code> ~570 QPS
	数据一致性完美验证

下一步建议:

- 1. 为 `/product/active` 的 `listActiveProducts()` 加 Redis 缓存 (TTL 10s)，降低 DB 压力
- 2. 若需更高下单吞吐，需要更多独立用户×商品组合 (生产环境天然满足)
- 3. 压测前务必预热 Redis 库存 —— 当前系统没有自动预热机制